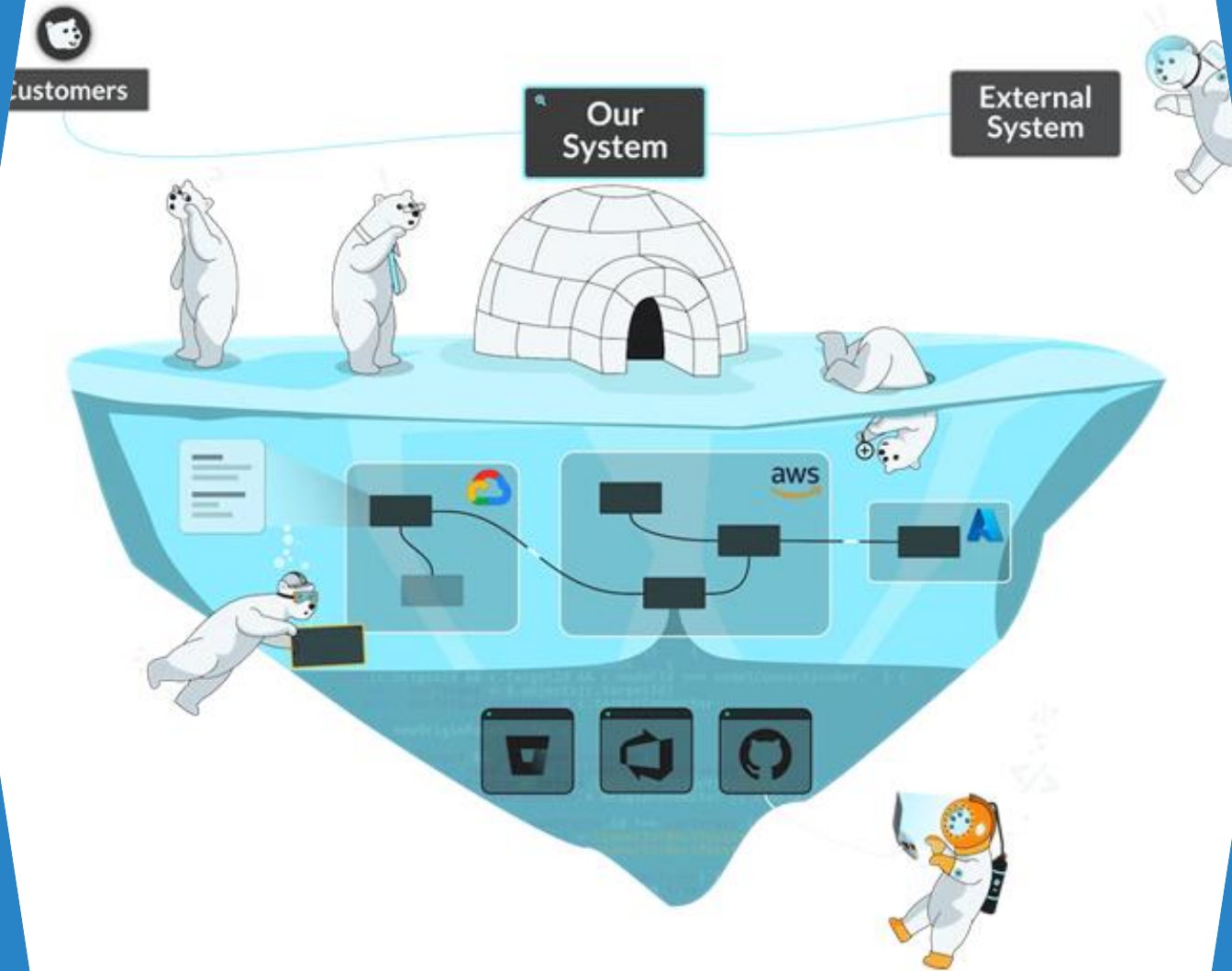


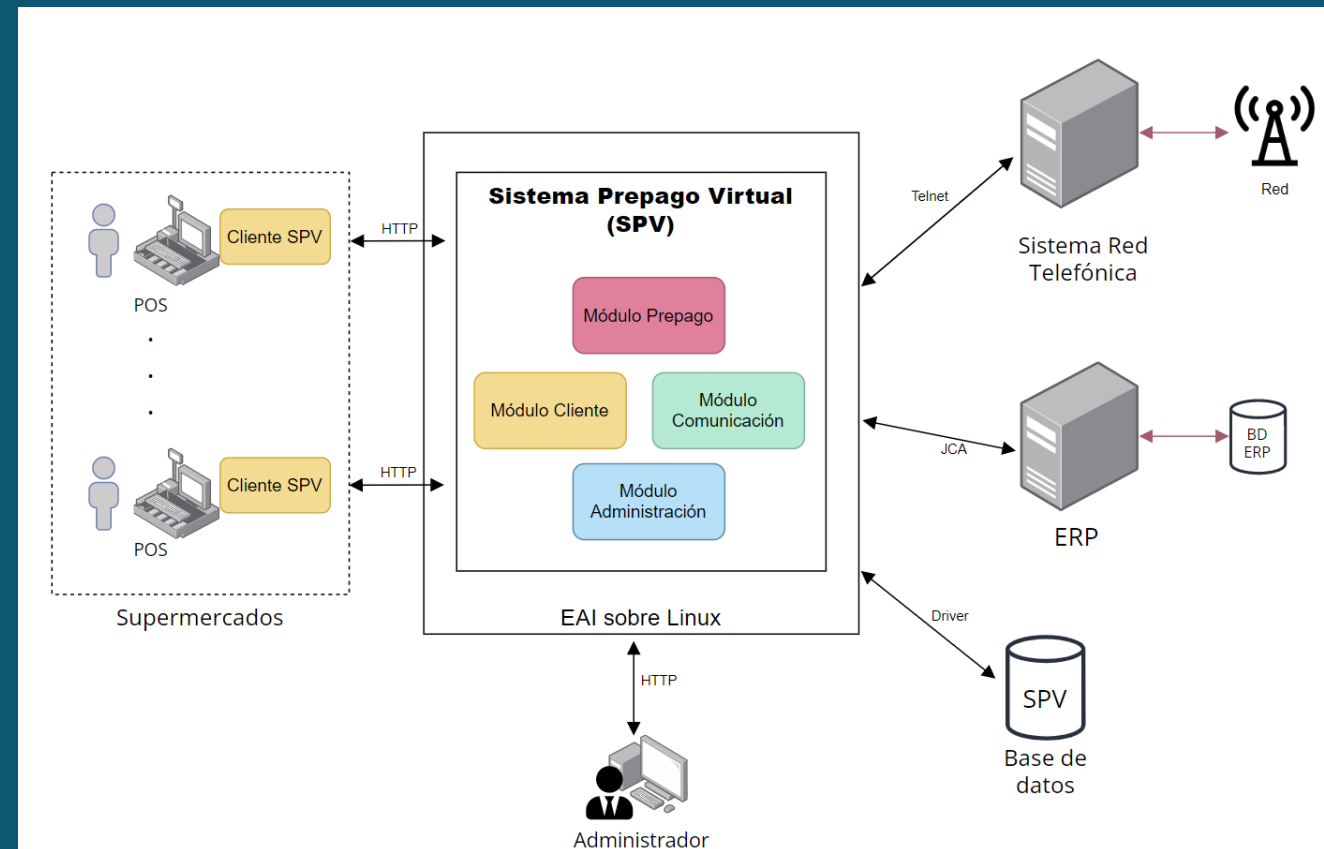
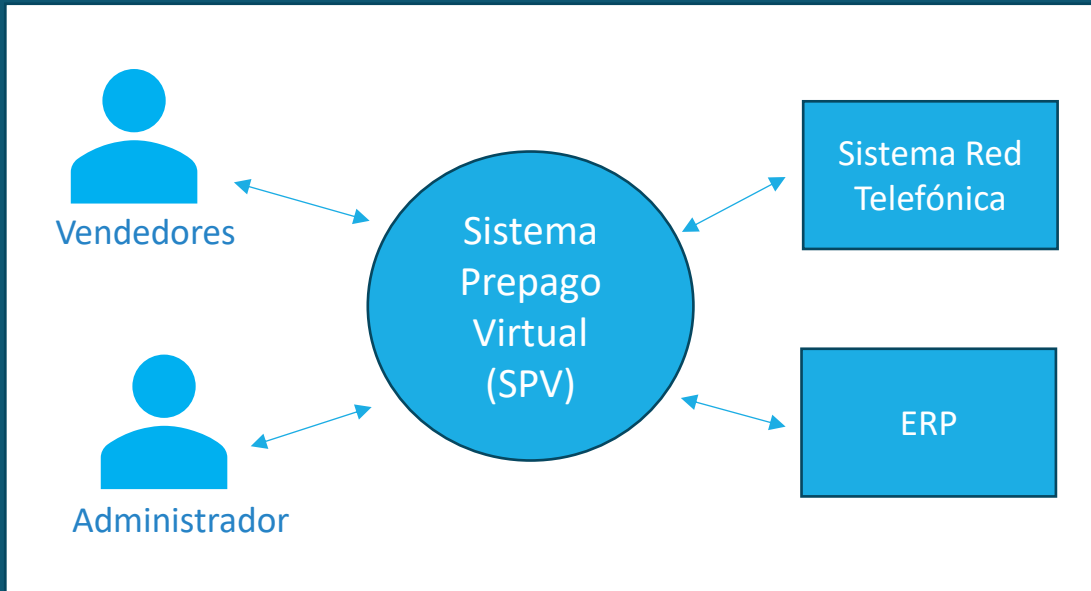


Documentación de la Arquitectura de Software

- Tendencias en Arquitectura de Software
 - Ciclo 2026-II
 - Hugo R. Cordero

No solo es el contexto y el diagrama general

- Documentar la arquitectura de un sistema no solo es el contexto
- Se acompaña el diagrama con la explicación resumida de cada elemento, sus responsabilidades, su justificación y su mecanismo de comunicación.
- La arquitectura no es un álbum de figuritas.



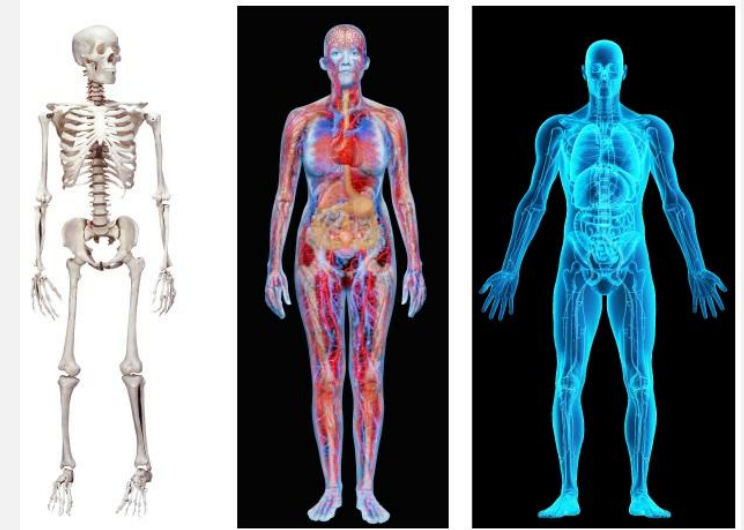
Plantilla para el Documento de Arquitectura



- Requerimientos no funcionales y restricciones para el sistema relevantes para la Arquitectura.
- Los elementos, que son los principales componentes del sistema y como se relacionarán
- Tecnologías elegidas para hacer frente a cada capa y las vistas que complementa la Arquitectura.

1. Descripción general
2. Drivers de la Arquitectura
3. Visión general de la Arquitectura
 - Contexto del sistema
 - Arquitectura general
 - Patrones arquitectónicos relevantes
 - Tecnologías seleccionadas
4. Vistas del sistema
5. Detalles de Implementación
6. Anexos

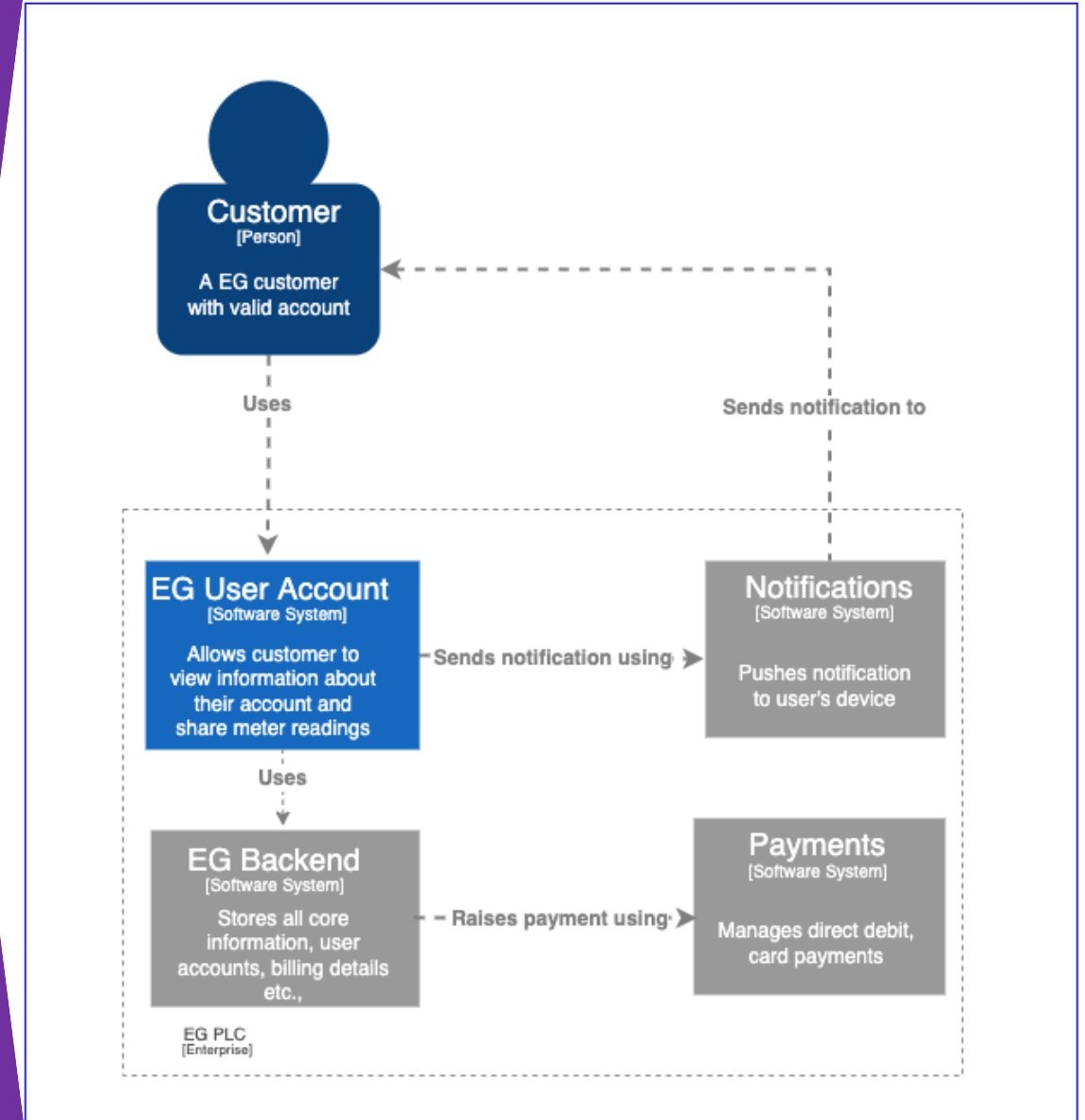
Las Vistas



- La arquitectura de un software está compuesta de varios puntos de vista: informativo o general, desarrollo, concurrencia, despliegue, operativo y funcional
- Una vista es una representación coherente de un conjunto de elementos arquitectónicos y las relaciones entre ellos (la estructura).
- Documentar una arquitectura es documentar las vistas relevantes y luego agregar la información que se aplica a más de una vista.
- Presentación (generalmente gráfica) con los elementos principales y sus relaciones, orientado a un público determinado. Tiene un catálogo explicando los elementos de la vista y sus propiedades.

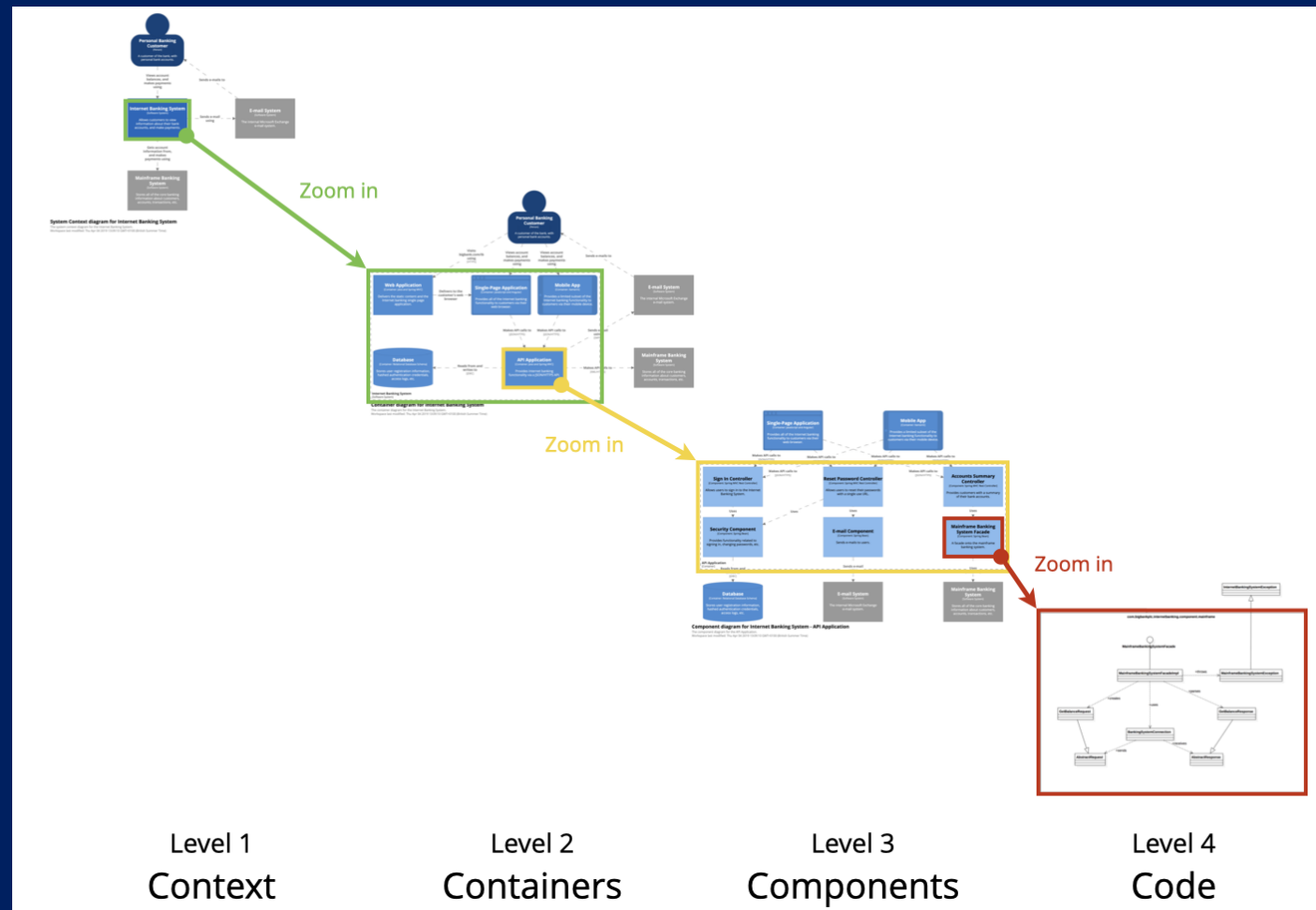
Modelos de Vistas

- El modelo 4+1 de Philippe B. Kruchten
 - Vista lógica
 - Vista desarrollo
 - Vista de procesos
 - Vista Física
 - Vista de Escenarios
- El modelo 4 vistas de Siemens (S4V) o de arquitectura de software de aplicaciones industriales de Dilip Soni, Robert L. Nord y Cristine Hofmeister.
 - Vista Conceptual
 - Vista de módulos
 - Vista de Ejecución
 - Vista de Códigos
- El modelo propuesto por el SEI
 - Vistas modulares
 - Vistas de componentes y servicios
 - Vistas de localización
- Modelo C4 propuesto por Simon Brown



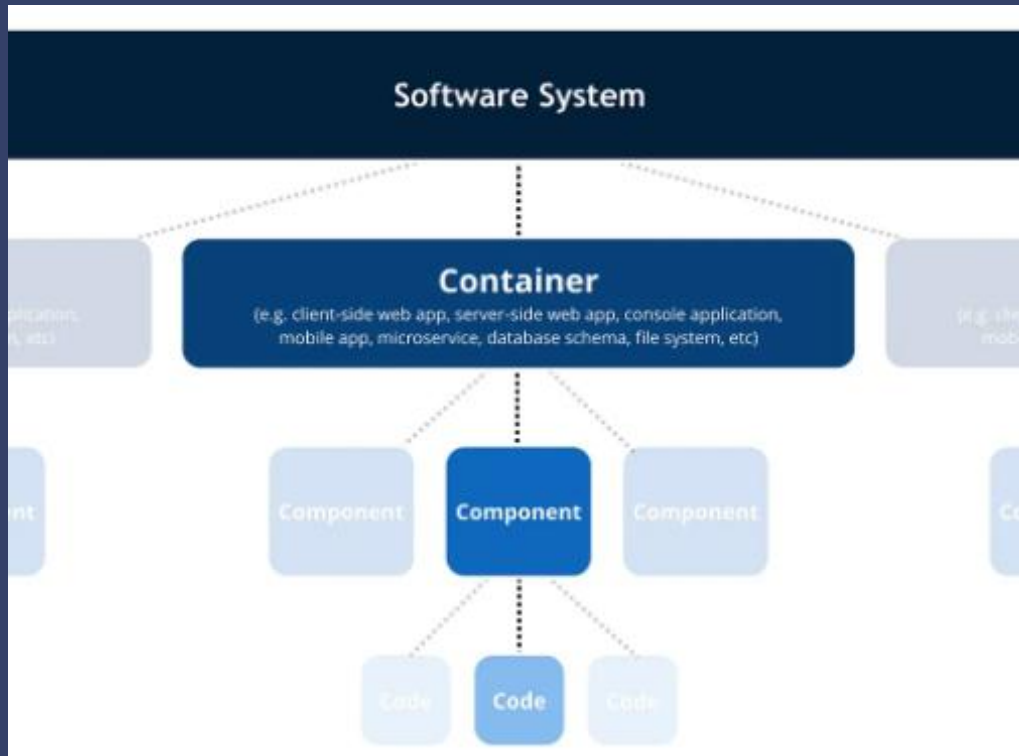
Modelo C4

- Nivel 1, contexto del sistema, muestra el sistema de software que está construyendo y cómo encaja en el mundo en términos de las personas que lo utilizan y los otros sistemas de software con los que interactúa.
- Nivel 2, diagrama de contenedores, amplía el sistema de software y muestra los contenedores (aplicaciones, almacenamiento de datos, microservicios, etc.) que componen este sistema de software.
- Nivel 3, diagrama de componentes, expande un contenedor individual para mostrar los componentes que contiene. Estos componentes deben asignarse a abstracciones reales (por ejemplo, una agrupación de códigos) en función de su código.
- Nivel 4: El código, si realmente lo desea o necesita, puede ampliar un componente individual para mostrar cómo se implementa ese componente.

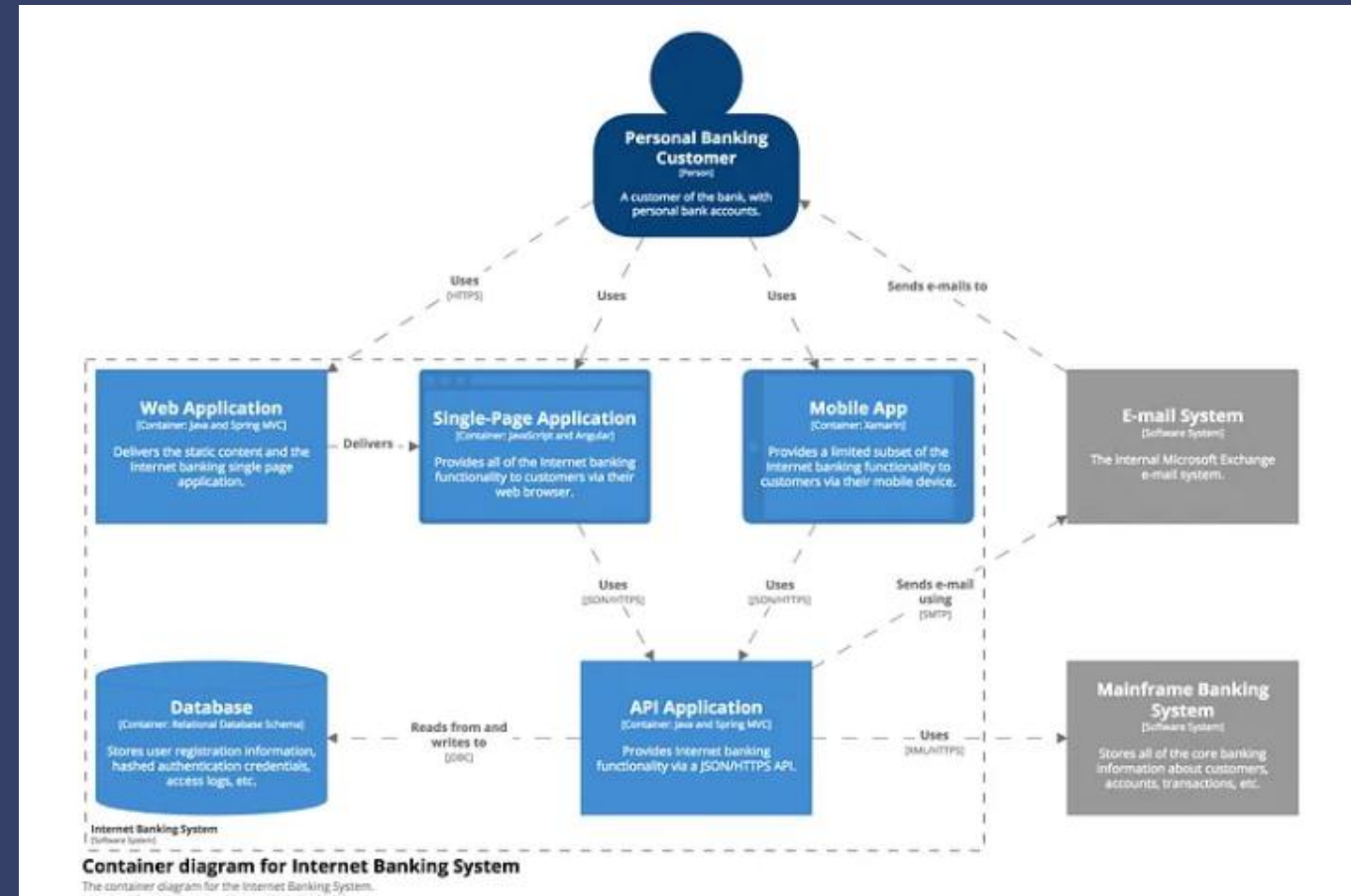


Vistas del modelo C4

- Jerarquía del modelo

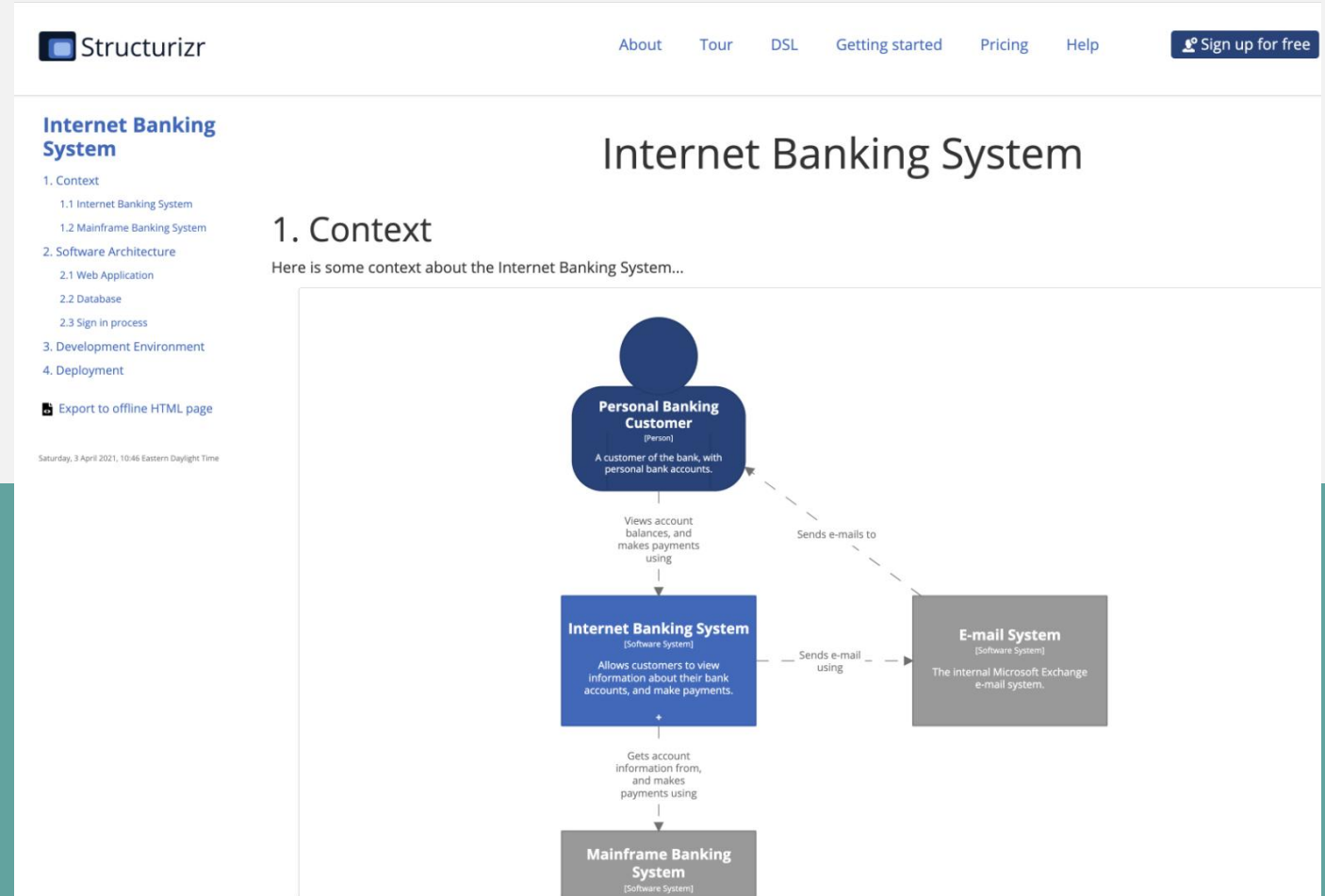


- El nivel 2, es el equivalente al diagrama general de la Arquitectura de Software
- Los diagramas del nivel 3, ya serían los equivalentes a las vistas de componentes, enfocando las distintas partes del sistema.



Herramientas para modelar la Arquitectura

- Herramientas de código abierto
 - diagrams.net
 - Archi
 - C4-PlantUML
 - Gaphor
- Herramientas comerciales de arquitectura
 - IcePanel
 - Astah
 - Structurizr
- Herramientas mainstream
 - Archipeg
 - Miro
 - Microsoft Visio



<https://structurizr.com/>

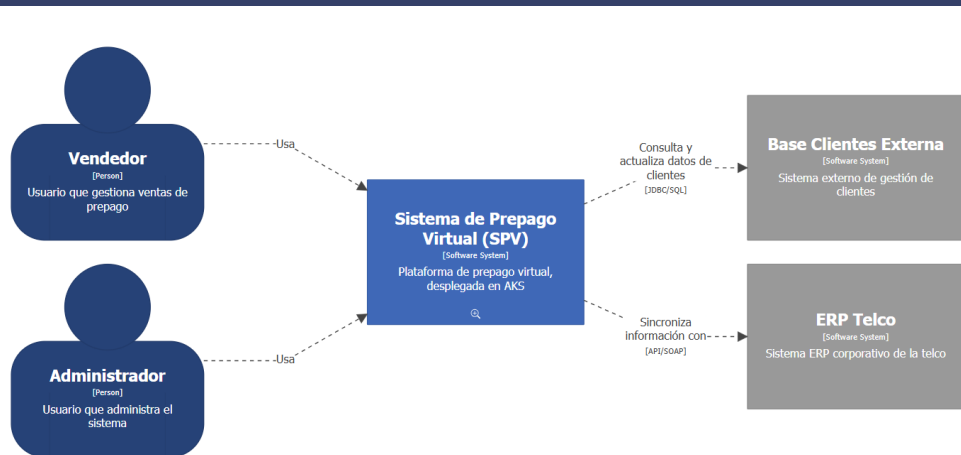
Structurizr:

arquitectura como código

<https://structurizr.com/>

Entornos para uso libre:

- Structurizr Lite
- <https://playground.structurizr.com/>



```
workspace "Sistema de Prepago Virtual (SPV)" "Diagrama de arquitectura del SPV" {

    !identifiers hierarchical

    model {
        vendedor = person "Vendedor" "Usuario que gestiona ventas de prepago"
        administrador = person "Administrador" "Usuario que administra el sistema"

        baseClientesExterna = softwareSystem "Base Clientes Externa" "Sistema externo de gestión de clientes" {
            tags "External" "Oracle"
        }

        erpTelco = softwareSystem "ERP Telco" "Sistema ERP corporativo de la telco" {
            tags "External" "SAP"
        }

        spv = softwareSystem "Sistema de Prepago Virtual (SPV)" "Plataforma de prepago virtual, desplegada en AKS" {
            tags "AKS"

            frontendVentas = container "Frontend Ventas" "Interfaz web usada por los vendedores" "React"
            frontendAdmin = container "Frontend Administración" "Interfaz web usada por los administradores" "Angular"
            apiGateway = container "API Gateway" "Punto de entrada único a los microservicios" "Azure API Gateway"
            backend = container "Backend de Microservicios" "Contiene la lógica de negocio del sistema" "Microservicios"
        }

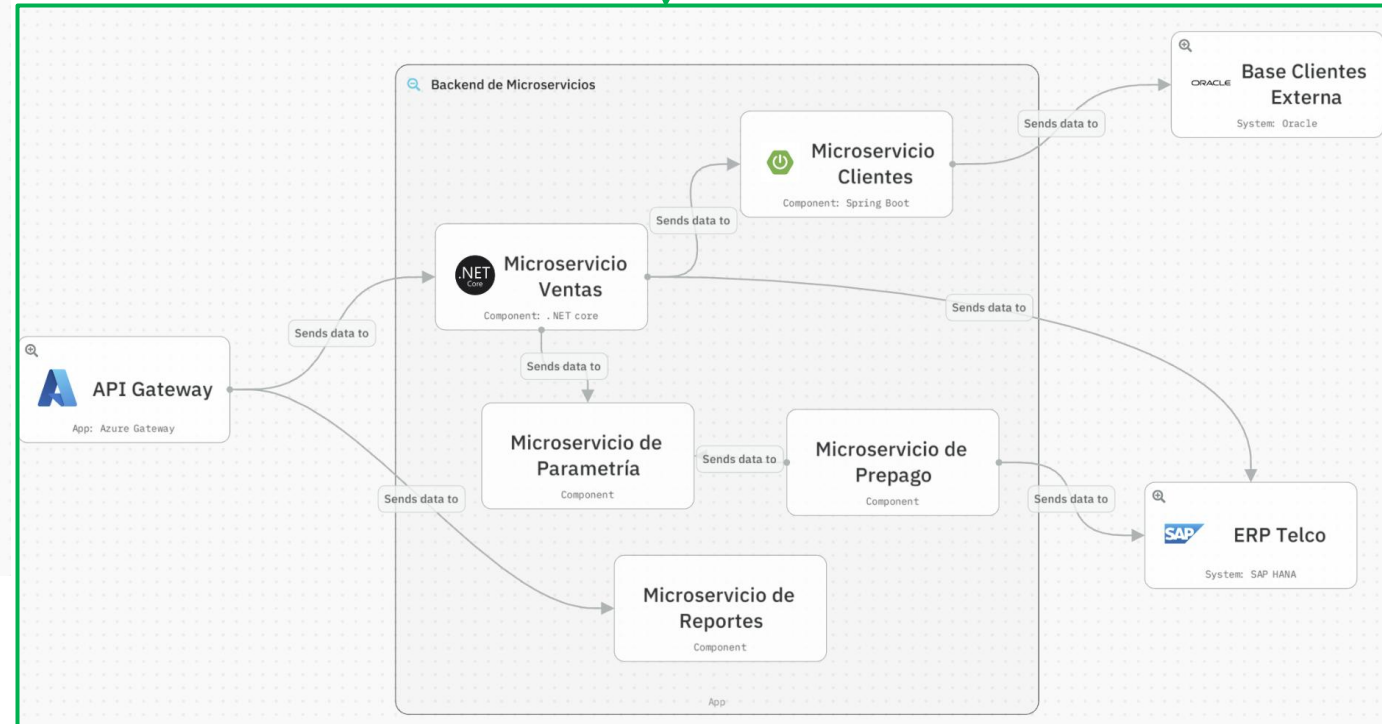
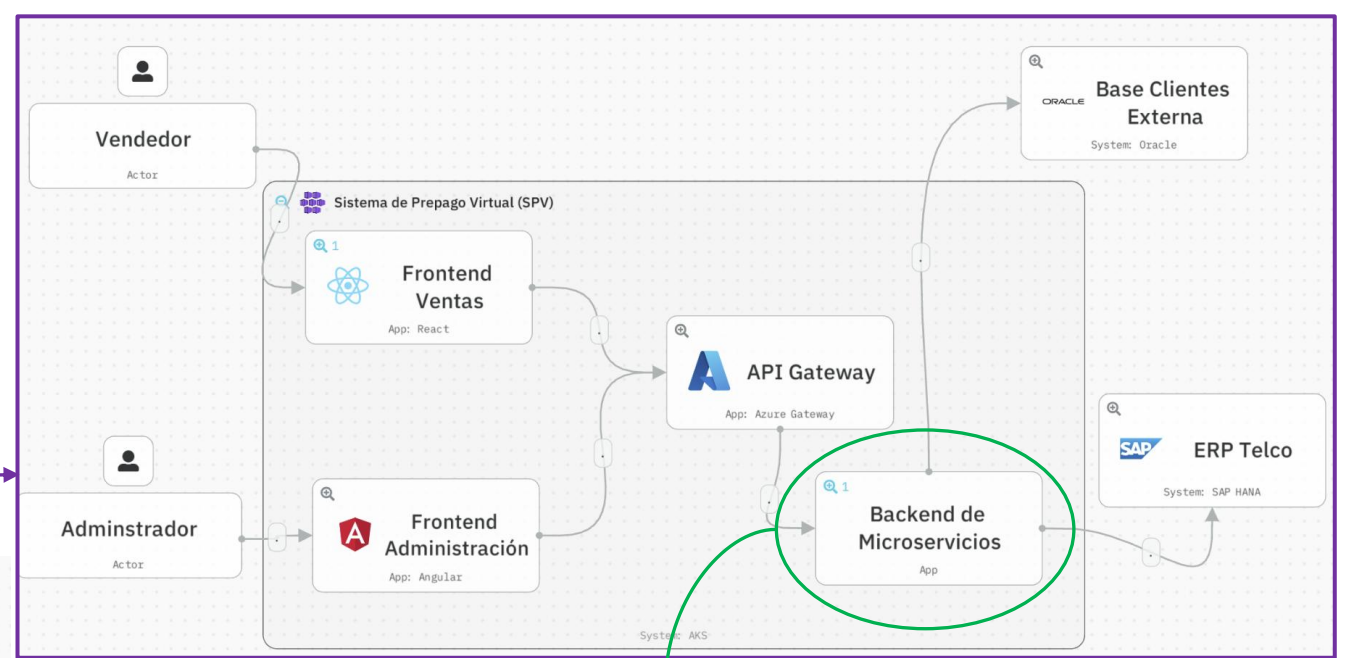
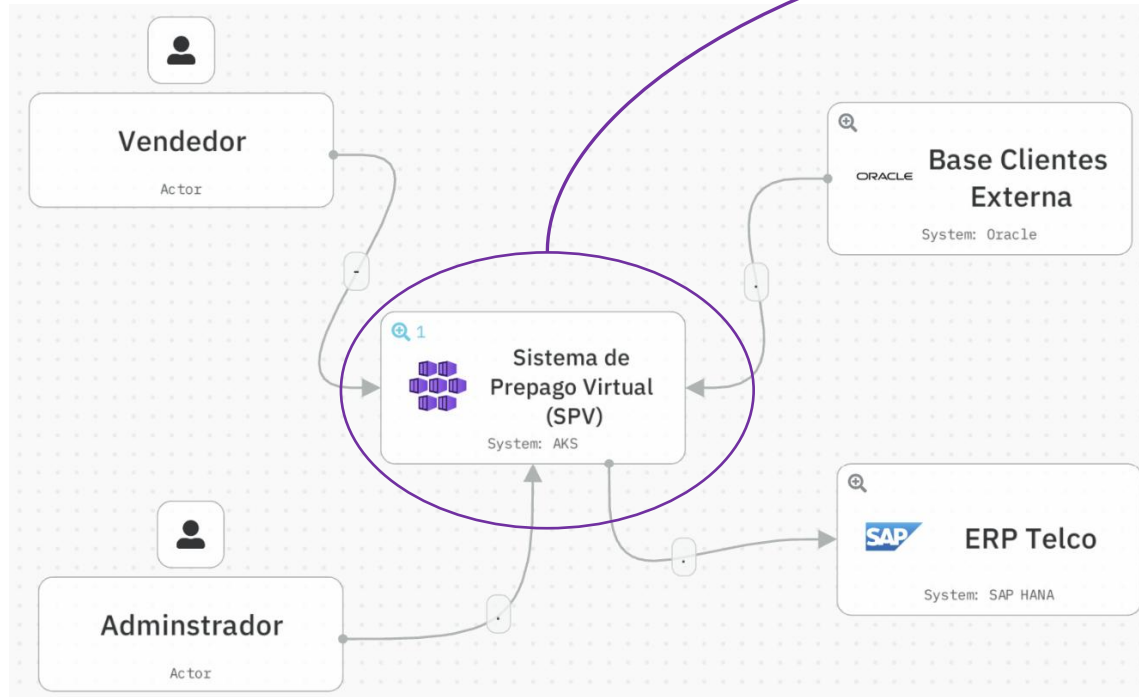
        # Relaciones
        vendedor -> spv.frontendVentas "Usa"
        administrador -> spv.frontendAdmin "Usa"
        spv.frontendVentas -> spv.apiGateway "Realiza peticiones API" "HTTPS/JSON"
        spv.frontendAdmin -> spv.apiGateway "Realiza peticiones API" "HTTPS/JSON"
        spv.apiGateway -> spv.backend "Enruta peticiones a" "HTTPS/JSON"
        spv.backend -> baseClientesExterna "Consulta y actualiza datos de clientes" "JDBC/SQL"
        spv.backend -> erpTelco "Sincroniza información con" "API/SOAP"
    }

    views {
        systemContext spv "SystemContext" {
            include *
            autoLayout lr
            description "Vista de contexto del Sistema de Prepago Virtual"
        }

        container spv "Containers" {
            include *
            autoLayout lr
            description "Vista de contenedores del Sistema de Prepago Virtual"
        }
    }
}
```

Modelo C4 con icePanel

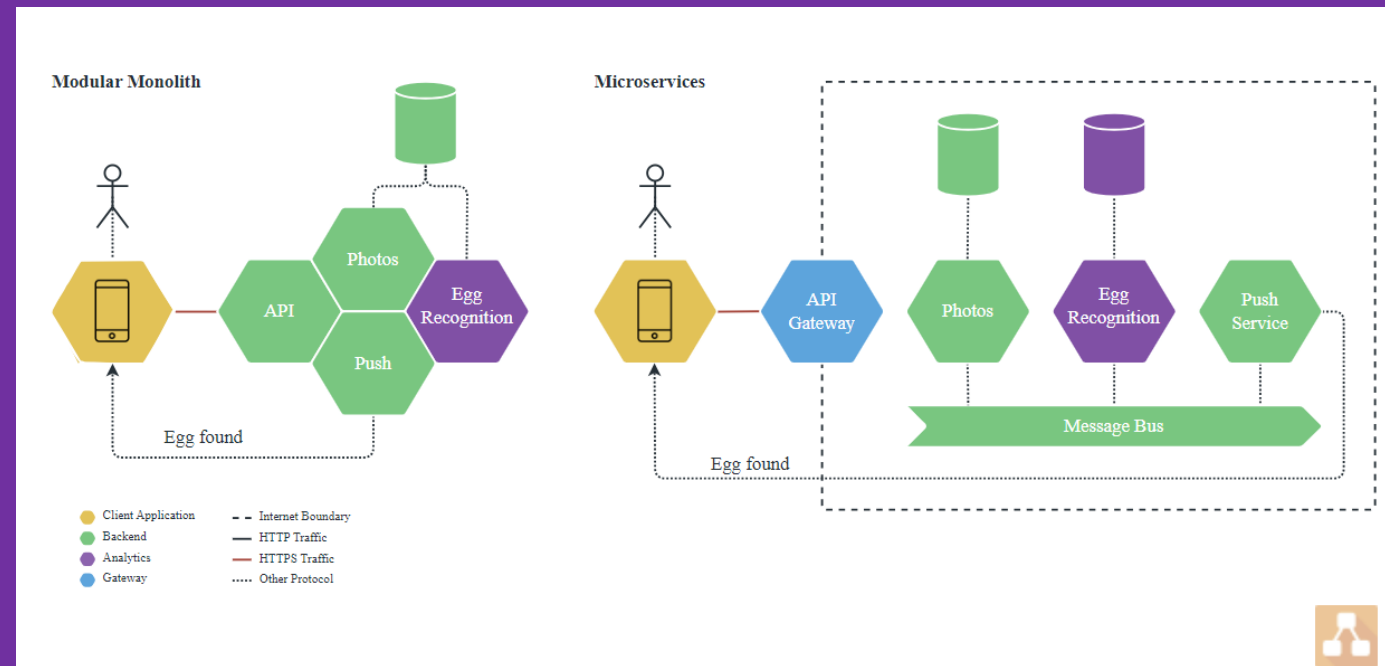
<https://icepanel.io/>



Registro de Decisiones Arquitectónicas (ADR)



- La arquitectura de los proyectos ágiles debe describirse y definirse de forma diferente.
 - No todas las decisiones se tomarán a la vez, ni tampoco **al comenzar el proyecto**.
- Los métodos ágiles no se oponen a la documentación, sólo a la documentación sin valor.
 - Nadie lee nunca los **documentos grandes**.
- Una de las cosas más difíciles de seguir durante la vida de un proyecto es la motivación que hay detrás de ciertas decisiones



Registro de decisiones Arquitectónicas

- Mantendremos una colección de registros para las decisiones “arquitectónicamente significativas”
 - Son las que afectan a la estructura, los requerimientos no funcionales, las dependencias, las interfaces o las técnicas de construcción.
- Un registro de decisiones de arquitectura (ADR) es un breve archivo de texto. Cada registro describe un conjunto de fuerzas y una única decisión en respuesta a esas fuerzas.
- Se puede generar ADRs desde decisiones ADD y se puede complementar el ADD con ADRs.

3. API Gateway como punto de entrada único al backend

Fecha: 2026-09-07

Estado

Propuesto

Contexto

Los dos frontends del SPV (Ventas y Administración) necesitan consumir la lógica de negocio expuesta por el Backend de Microservicios. Exponer cada microservicio directamente a los frontends dificultaría la gestión de seguridad, versionado y evolución del backend.

Decisión

Se introduce un ****API Gateway**** (Azure API Gateway) como único punto de entrada entre los frontends y el Backend de Microservicios. Ningún frontend se conecta directamente a los microservicios internos.

Consecuencias

****Positivas****

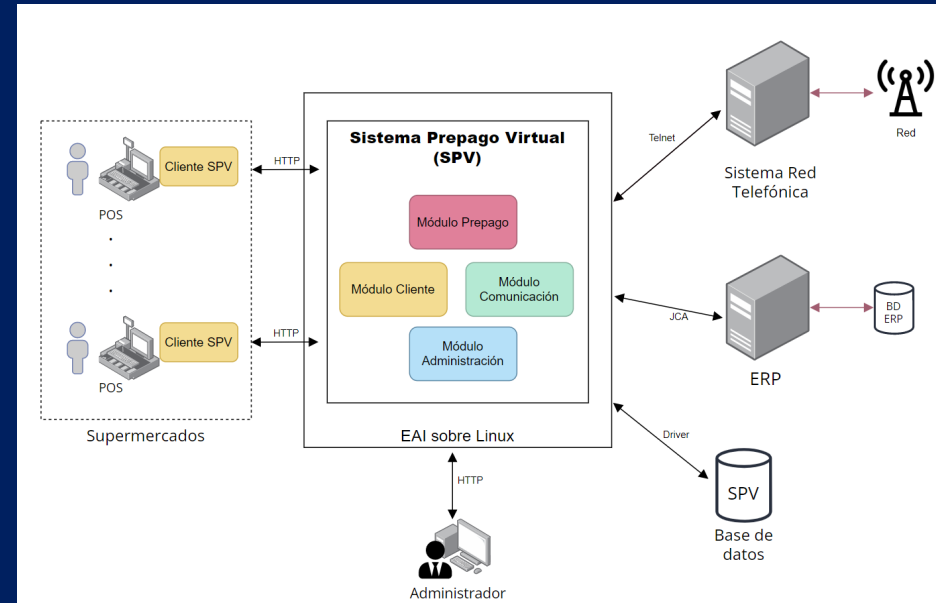
- Centraliza aspectos transversales como autenticación, autorización, rate limiting y ruteo hacia los microservicios correspondientes.
- Oculta la complejidad y la topología interna del backend a los frontends, permitiendo reorganizar microservicios sin romper a los clientes.
- Al usar Azure API Gateway, se aprovecha el mismo ecosistema cloud (Azure) donde se despliega el resto del sistema.

****Negativas****

- Introduce un componente crítico: si el Gateway falla o no está bien dimensionado, afecta a todo el sistema (single point of failure si no se implementa con alta disponibilidad).
- Agrega un salto de red adicional (latencia) entre los frontends y el backend.

La documentación de la Arquitectura y la IA

- IcePanel publicó "The model code gap" donde recomienda no generar el diagrama automáticamente analizando el código fuente o infraestructura real. Argumentan que un diagrama generado 1:1 desde el código es "preciso pero inútil", termina con demasiado detalle de bajo nivel y pierde el propósito de comunicar arquitectura a nivel humano.
- La IA hoy está en el rol de traductor entre una representación humana (diagrama, conversación, decisión tomada) y una representación estructurada y versionable (DSL, JSON, Markdown de ADR), no en leer todo el código crudo para inventar una arquitectura.



```
workspace "Sistema Certificación Global para Ingeniería (GECS)" "Diagrama de contexto del sistema GECS" {
    model {
        adminUniversity = person "Admin University" "Mantiene información de la universidad, exámenes, participantes, evaluaciones, resultados y certificaciones." {
            tags "UniversityAdmin"
        }

        engineer = person "Engineer" "Realiza exámenes de certificación y consulta sus resultados y certificados." {
            tags "PlatformUser"
        }

        adminPlatform = person "Admin Platform" "Habilita nuevas universidades y gestiona los accesos de los administradores." {
            tags "PlatformUser"
        }

        geecs = softwareSystem "Sistema Certificación Global para Ingeniería (GECS)" "Sistema Certificación Global para Ingeniería (GECS)"

        twilio = softwareSystem "Twilio" "Servicio para envío de SMS y WhatsApp" {
            tags "External" "Twilio"
        }

        sendgrid = softwareSystem "SendGrid" "Servicio para envío de correo electrónico" {
            tags "External" "SendGrid"
        }

        # Relaciones
        adminUniversity -> geecs "Mantiene información de la universidad, exámenes, participantes, evaluaciones, resultados y certificaciones."
        engineer -> geecs "Realiza exámenes y revisa sus resultados y certificados"
        adminPlatform -> geecs "Habilita nuevas universidades y accesos a los administradores"
        geecs -> twilio "Notificar por SMS y WhatsApp"
        geecs -> sendgrid "Notificar por correo electrónico"
    }
}
```

- Se compartirá un caso práctico en clase para que pueda realizar la documentación de la Arquitectura.
- Elabore los diagramas del modelo C4 en Structurizr o icePanel.
- Identifique las tres principales decisiones de diseño y elabore sus ADR.

Llamada a la acción



Tarea 1

- Para su proyecto desarrolle:
 - Las vistas del modelo C4 y documento en <https://icepanel.io/>
 - Elabore el DSL y genere los mismos diagramas en <https://playground.structurizr.com/>
 - Documente al menos 7 ADRs que condujeron a la Arquitectura y complemento a los diagramas anteriores.
 - Investigue sobre adr-tool o Log4brains, y utilícelo para registrar sus ADRs.





Gracias



Correo electrónico
hcorderos@unmsm.edu.pe



X / Threads
@hugorcorderos